

SoK: A Beginner-Friendly Introduction to Fault Injection Attacks

Christopher Simon Liu*
The Ohio State University
liu.8689@osu.edu

Fan Wang*
The Ohio State University
wang.13244@.osu.edu

Patrick Gould
The Ohio State University
gould.279@osu.edu

Carter Yagemann
The Ohio State University
yagemann.1@osu.edu

Abstract—Fault Injection is the study of observing how systems behave under unusual stress, environmental or otherwise. In practice, fault injection involves testing the limits of computer systems and finding novel ways to potentially break cyber-physical security.

The contributions of this paper are three-fold. First, we provide a beginner-friendly introduction to this research topic and an in-depth taxonomy of fault injection techniques. Second, we highlight the current state-of-the-art and provide a cost-benefit analysis of each attack method. Third, for those interested in doing fault injection research, we provide a replication analysis of an existing vulnerability detection tool and identify a research focus for future work.

1. Introduction

The most publicly known example of fault injection in computer systems would be in 2013 when an ionizing particle caused a Super Mario “speed runner” to complete in record time [1]. A more serious example is an incident in 2004 when a cosmic ray caused an election voting system to count 4096 extra votes [2]. Needless to say, fault injections lead to unintended code values and unintended code executions. If these faults are intentional, they are a significant cyber threat; Fault injections are active physical attacks.

The oldest research paper on the topic of fault injection was in 1975, observing how cosmic rays cause unexpected triggering of digital circuits in satellites [3]. In 1997, a theoretical model demonstrated that it was possible to compromise encrypted systems with hardware faults [4]. This seminal work sparked a substantial body of research focused on exploiting hardware vulnerabilities.

Since then, many fault injection techniques have been created, researched, and published. Of the various injection techniques, voltage glitching [5]–[8], clock glitching [9], [10], laser fault injection [11], [12], and electromagnetic fault injection [13], [14] are among the most prevalent.

One way to describe fault injection is to compare it with “torturing” computer chips until they submit or reveal secrets [15]. From the perspective of an attacker, fault injection techniques are dangerously useful for bypassing secure boot [16], enabling arbitrary code execution [17], escalating privileges [18], and extracting keys [19].

When studying fault injections, we separate the process into two stages: Fault Modeling (examining attack methods) and Fault Analysis (examining faulty outcomes). The following is a list of fault models used in the literature, providing an overview of how fault injection is used.

Bit flip Reverse Engineering Neural Networks [20], Adversarial Neural Networks [21]

Bit set/reset Breaking Substitution-Permutation Network (SPN) Block Ciphers [22]

Random byte Breaking Encryption with Differential Fault Analysis (DFA) [23]

Instruction skip Escalating Privileges [24], Extracting Keys [25], Neural Network Misclassifications [26]

Execution faults Breaking Physically Unclonable Functions (PUFs) [27]

Stuck-at faults Breaking Symmetric Cryptography [28], Breaking Random Number Generators (RNG) [29]

In response to Fault Injection Attacks (FIA), various hardware-based tools [30]–[33] and software-based tools [34]–[40] were created to assess device vulnerability.

In this paper, we focus on the software-level manifestation of injected faults. We complete an examination and replication analysis of an existing software-based FIA vulnerability detection tool. We perform a literature review to compare varying fault models. We identify limitations in these existing fault models. We then highlight an area for further research.

Despite FIAs involving both hardware (for inducing) and software (for exploiting), existing work focuses exclusively on one or the other. We hypothesize that the identified limitations are addressed by integrating both, bridging the gap in FIA modeling.

In summary, we make the following contributions.

- We provide a systematic review of existing work.
- From our review, we identify a critical gap in fault modeling between hardware and software.
- We propose a synthesis solution to identified limitations to motivate future work.

*. The two lead authors contributed equally to this work.

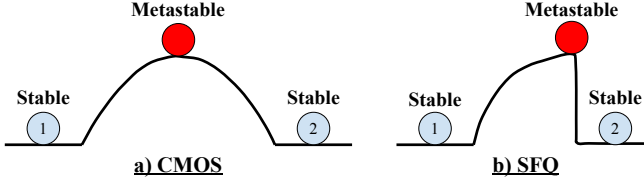


Figure 1. Mechanical Metastability: The Ball on a Hill in [41]

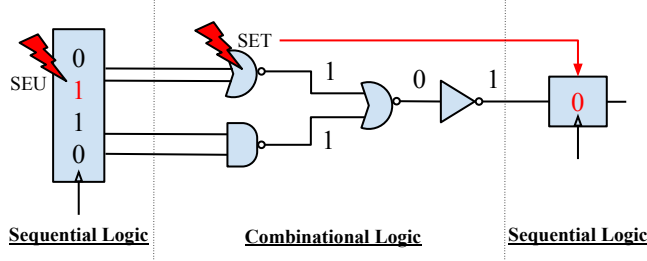


Figure 2. Single Event Upset and Single Event Transient Effects in [43]

2. Foundations

This section provides the essential knowledge for understanding FIAs. We show Metastability’s role in device vulnerability and how it leads to FIA through Single Event Upsets and Transient Effects.

2.1. Metastability

Metastability is an unstable state in electronics that leads to system failure when it occurs in integrated circuits (ICs). A useful metaphor (Figure 1) for this phenomenon is a ball positioned at the top of a hill; it may eventually fall into one of two adjacent valleys, but it is uncertain which valley it will end up in. Similarly, every bistable storage element in an IC may resolve to a ‘0’ or ‘1’. The metastability of a storage element is induced by a voltage level that lies between the logical states of ‘0’ and ‘1’.

Faulty outputs caused by metastability propagate through ICs, leading to more significant errors. However, it is impossible for a digital circuit to completely detect or prevent metastability [42]. Fault injection techniques, such as voltage and clock glitches, increase the probability of metastability occurring.

2.2. Single Event Upset

Single Event Upsets (SEU) are changes in the state of a transistor, altering the logic of a digital circuit. This leads to a Single Event Transient (SET) effect where an incorrect result is interpreted as the final value, leading to the propagation of errors and a fault injection (Figure 2).

As mentioned in the introduction, the first documented occurrence of SEU was reported in 1975, where four upsets were observed during 17 years of satellite operation in

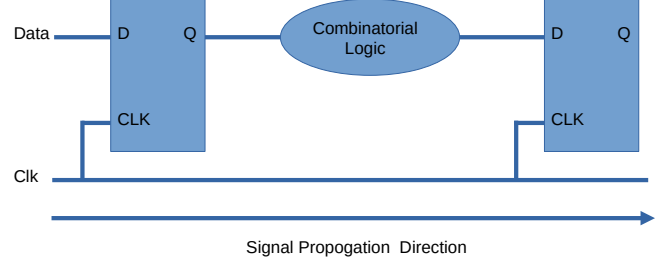


Figure 3. Combinatorial Logic

space [3]. As the occurrence of SEUs became more evident, the hardening of integrated circuits (ICs) emerged as a primary mitigation strategy, proposed by several research groups [44], [45]. Further research revealed that the ionization processes responsible for SEUs vary [46], [47]. Today, there is growing interest in understanding the impact of SEUs on machine learning systems, as machine learning has become increasingly prominent [48], [49].

3. Attack Taxonomy

This section classifies each fault injection technique or “attack vector”. We show that the underlying mechanism behind attack methods are “Timing Constraint Violations” as well as the Precision vs. Cost trade-off of each method.

3.1. Timing Constraint Violations

Timing constraints are fundamental requirements for the timing of digital ICs that must be met to ensure correct functionality. Most digital ICs are generalized by an architecture in which combinatorial logic is located between two D flip-flops (Figure 3). The correctness of the data latched into the downstream D flip-flop is critically dependent on avoiding timing violations. Refer to Appendix B for the specific parameters of timing constraints.

In short, when timing constraints are violated, the setup time requirement will no longer be satisfied (Figure 4). This leads to incorrect values being set, errors propagated, and a Fault Injection Attack on the circuit.

The underlying mechanism behind voltage glitches is hypothesized to be caused by metastability resulting from timing constraint violations [50]. This hypothesis was substantiated by later experiments, finding that the faults produced by clock glitches and voltage glitches were identical [51]. We therefore lump together Clock and Voltage Glitching because their method is *functionally the same*.

Similarly, the underlying mechanism behind Electromagnetic Fault Injection (EMFI) is theorized to be the violation of timing constraints [52]. Another explanation behind EMFI is the disruption of the sampling process of D flip-flops [53]. Experiments have shown that both mechanisms may coexist during an EMFI attack [54].

Additionally, laser fault injection directly flips individual bits with ionizing radiation. This alters the state of a logic gate, leading to errors in the output [55].

3.2. Voltage Glitch

Voltage glitches involve briefly changing the supplied voltage of a device outside the specified intended range. Imagine switching a device off and on before it can properly react. This is voltage glitching.

Voltage glitches have two primary effects on digital circuit combinatorial logic. The first effect occurs directly on the victim gate, and the second arises from the propagation of the error through logic [50]. In this paper, we adopt the same classification to analyze the impact of voltage glitches on Integrated Circuits (IC).

Refer to Appendix C for details of how changes in voltage break timing constraints.

3.2.1. Underpowering. As the name suggests, underpowering supply glitches result from lowering the IC's voltage level below the nominal value. Underpowering has been widely utilized in FIAs [5], [7], [51], [56] [57]. Multiple experiments have substantiated underpowering by connecting a device under test (DUT) to a remotely controlled power supply [5], [7].

Considerations: It is evident that underpowering is relatively easy to carry out and cost-effective. It also comes with significant drawbacks. It is challenging to precisely inject faults into a DUT with this method, given that the supply voltage affects the entire IC.

3.2.2. Negative Power Supply Glitch. Similar to underpowering, negative power supply glitches result from a precise drop in voltage. A negative power supply glitch typically lasts for a shorter duration with a sharp voltage waveform, whereas the waveform of underpowering is flatter [6]. Negative power triggers a glitch at the exact moment when the desired round of computation begins, allowing for more targeted fault injection [7].

Considerations: It is a more precise form of underpowering. It requires a deeper understanding of the IC and the specific processes running on it, making it more difficult.

3.2.3. Positive Power Supply Glitch. At first glance, the concept of a positive power glitch may seem counterintuitive, as an increase in supply voltage typically results in a decrease in data propagation time. However, a positive power glitch induces a damping oscillation in the voltage supply, indirectly causing the voltage to drop below the nominal level, leading to timing constraint violations and faults [5].

Considerations: It is an unintended negative power glitch. It is not widely utilized in FIAs.

3.3. Clock Glitch

Clock glitches involve injecting short signals that deviate from the nominal clock signal frequency. Clock glitches share the same underlying mechanism as voltage glitches [51]. Since this has already been discussed previously, we will keep our explanation here brief.

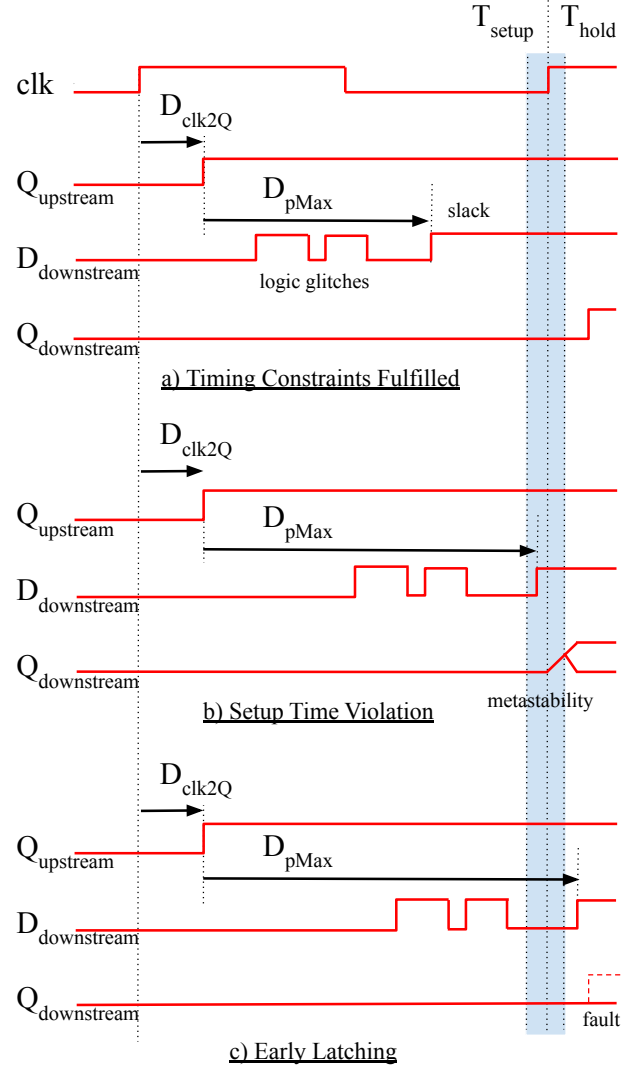


Figure 4. Voltage Attack Waveforms in [5]

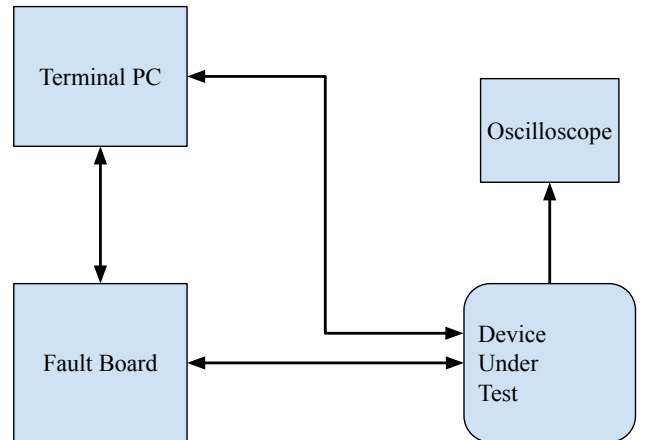


Figure 5. Diagram of the Clock Glitch Environment in [58]

Injecting a faulty clock signal triggers incorrect execution by violating timing constraints. The challenge in this process is how to generate the glitches. The use of precise clock glitches requires several preconditions due to the high precision needed in signal generation. This has led to a divergence in methodologies for generating clock glitches.

3.3.1. Precisely Timed Glitch. As the name suggests, this approach depends on generating precise and deterministic clock signals. To satisfy these conditions, the target device should have already been well studied before. Existing experiments demonstrate that precisely timed clock glitches are highly reproducible with 100% reliability. [10]. Glitches are injected after a predefined number of clock cycles, calculated by a FPGA fault board [59]. Other experiments, like the one shown in (Figure 5), also show the reproducibility of clock glitch FIAs [58].

Considerations: It is deterministic (not random), reproducible, and reliable. It is *a lot* of work to precisely time.

3.3.2. Fuzzy Glitch. In contrast to precisely timed clock glitches, fuzzy-glitch techniques generate randomized clock signals. A fuzzy clock signal generator, utilizing two ring oscillators operating at different frequencies, produces a fuzzy glitch signal, which is both random and high-frequency [9]. Fuzzy glitches are shown to achieve a 75% success rate under optimal conditions, with a remaining success rate above 50% across a wide range of parameters [9].

Considerations: It is non-deterministic (random). It is easier than calculating the precise time. Similar to traditional “fuzzing”, it takes a brute-force approach in finding both useful and useless inputs.

3.4. RowHammer

This is a voltage glitch in memory instead of the processor. While technically a software attack, the underlying cause of a RowHammer Attack is the leakage of charge in certain DRAM cells, triggered by voltage fluctuations.

RowHammer was first discovered in 2014 [8]. As technology advanced, the density of DRAM cells has increased, which in turn has made these cells more susceptible to errors due to their close proximity. To execute a RowHammer attack, the wordline must be toggled on and off repeatedly, which is achieved by reading the same DRAM row multiple times. Refer to Figure 6 for layout.

Repeatedly toggling the “wordline” activation wire’s voltage causes some cells in neighboring rows to leak charge at an accelerated rate. This leakage prevents the cells from retaining their charge for the standard refresh interval, leading to data loss and disturbance errors [8]. Experiments have shown that this is achieved in as few as 139,000 reads to a DRAM address [8].

Considerations: It is technically a voltage glitch in memory instead of the processor which does not require physical access. It is the only attack method that is completely software-based. It still has challenges in placing the target data in specific physical memory locations that are vulnerable to fault injection.

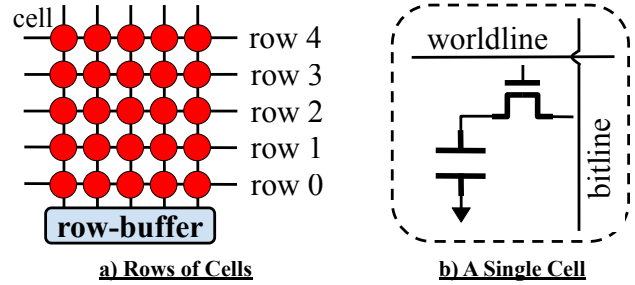


Figure 6. DRAM Organization in [8]

3.4.1. Deterministic RowHammer. In 2016, the deterministic RowHammer attack was proposed, targeting ARM/Android systems [60]. This explored the feasibility of using RowHammer in modern always-on commodity mobile systems. In addition, the attack used only standard features, reducing the baseline requirements for a successful attack.

To ensure that sensitive data reside within exploitable memory components, deterministic memory massaging techniques are employed. For example, “Phys. Feng Shui” is a method of positioning sensitive data in physically vulnerable memory addresses, utilizing predictable memory reuse patterns and occupying unused memory chunks [60]. Other techniques include Memory Deduplication to make the RowHammer attack deterministic [61].

Considerations: It focuses on precise memory locations that are certain to be vulnerable.

3.4.2. Probabilistic RowHammer. In 2016, it was also found that the use of non-temporal store instructions, where data is created and not immediately consumed, triggers RowHammer vulnerabilities [62]. This means that benign code, which do not follow the normal cache-coherency rules, can sometimes cause a RowHammer event.

To ensure that sensitive data reside within exploitable memory components, probabilistic memory massaging techniques are also employed. Memory Spraying is a method in which the target memory is filled with page tables [63]. Due to the probabilistic nature of this method, not every mapped page table is physically vulnerable to RowHammer. However, the specific layout of the sprayed physical memory can increase the likelihood that sensitive page tables will be located in a vulnerable position. Other techniques include virtualization to achieve a similar goal [64].

Considerations: It focuses on random memory locations that are potentially vulnerable.

3.5. Laser Fault Attack

Laser fault injection (Laser FI) is a powerful yet costly fault injection technique that uses ionizing radiation to flip circuit logic. Imagine creating an intentional cosmic ray. This is laser fault injection. The Laser Fault Attack was first introduced in 2002 [65], inspired by earlier experiments in ionizing radiation on semiconductors [66].

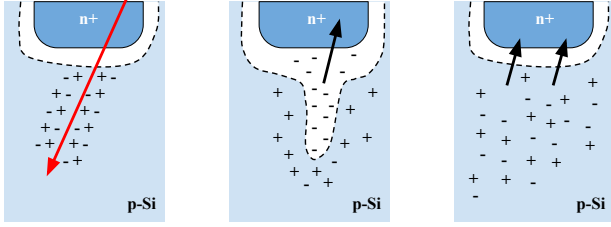


Figure 7. Physical effect of Laser Fault Injection in [68]

One significant advantage of laser fault injection over other methods is its ability to inject faults into a limited area. Voltage and clock glitch methods often affect entire systems, whereas a laser targets a specific, small area, allowing for precise control over the fault injection. Photoelectrical Laser Stimulation (PLS) generates photoelectric currents in specific cells [67]. As shown in Figure 7, this causes a reduction in voltage, which changes values [68].

Despite the shrinking node sizes of Moore’s Law, numerous studies have demonstrated the continued feasibility of injecting single-bit or single-byte faults into modern ICs with Laser FI. Experiments in 2010 on an 8-bit microcontroller with a 0.35 μm technology node show that it remains practical to perform single-bit or single-byte faults [69]. Laser FI remains feasible and reproducible with careful beam tuning and timing [70]. Later experiments in 2016 inject into 45 nm technology nodes, concluding that individual bit faults are feasible for both 90 nm and 45 nm chips, although the latter presents certain limitations [11]. Additional studies have further validated the feasibility of single-bit fault injection using laser fault attacks [12], [71].

3.5.1. Controllable Laser Fault Injection. How are laser-injected faults fully controlled? Specialized lasers. Fine-tuning the parameters of laser fault injection results in perfect faults: able to switch any register value between ‘0’ and ‘1’ by adjusting the location [72]. Experiments in 2020, as shown in Figure 8, demonstrate that laser faults enable controllable instruction replacement, circumventing existing countermeasures [73].

Considerations: It requires a decapsulated IC and removing the protective covering of an IC is an expensive, labor intensive board modification. It also has limitations based on node size. It is extremely precise.

3.5.2. Low-Cost Laser Fault Injection. The first laser fault injection used an inexpensive \$30 second-hand flashgun and an \$8 laser pointer to cause random faults [65]. On the other side of the spectrum, a million dollar nano-focused X-ray beam was used to target a single transistor [74]. This has motivated research into high-precision and low-cost laser fault injection [75].

Considerations: It is still more expensive compared to the previously discussed techniques. It is becoming more affordable.

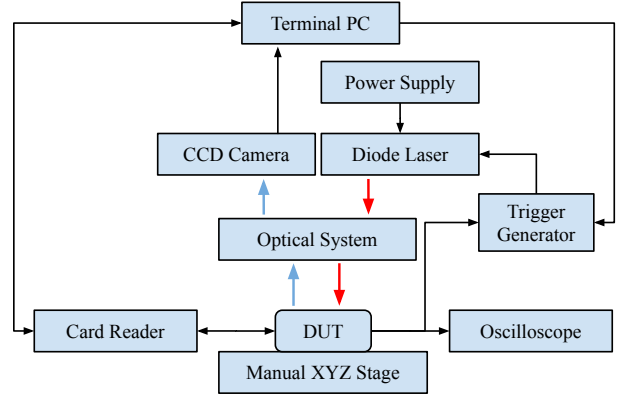


Figure 8. Diagram of the Laser Setup in [73]

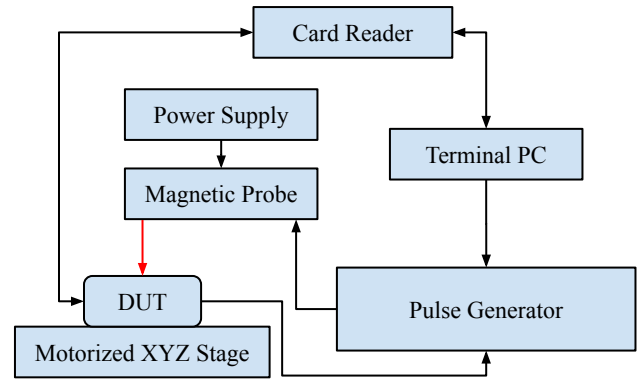


Figure 9. EM Pulse Injection Bench in [76]

3.6. Electromagnetic Fault Injection

Electromagnetic fault injection (EMFI), as shown in Figure 9, involves blasting an IC with a brief burst of electromagnetic energy from closely placed magnetic coils (Magnetic Probe) [76]. Imagine sending a wave of electrons at an electronic device. If it is not destroyed, this is EMFI. The Electromagnetic Pulse (EMP) Fault Attack was first introduced in 2012 [76]. The resulting EM disturbance generates a highly localized transient voltage within the IC, altering the circuit’s behavior [76]. Subsequent work further confirmed the locality of EM-generated faults and demonstrated the ability to induce both bit-set and bit-reset faults [77]. Unlike Laser FI, many studies have successfully designed low-cost EM probes for EMFI attacks [78]–[80].

Considerations: It does not require board modification, unlike Laser FI. It is cheaper. It is also very precise, but it will not target a single bit or transistor. It has a local effect.

3.7. Taxonomy Chart

After performing a systematic review of existing work on FIA attacks, we constructed a taxonomy chart. See Figure 10.

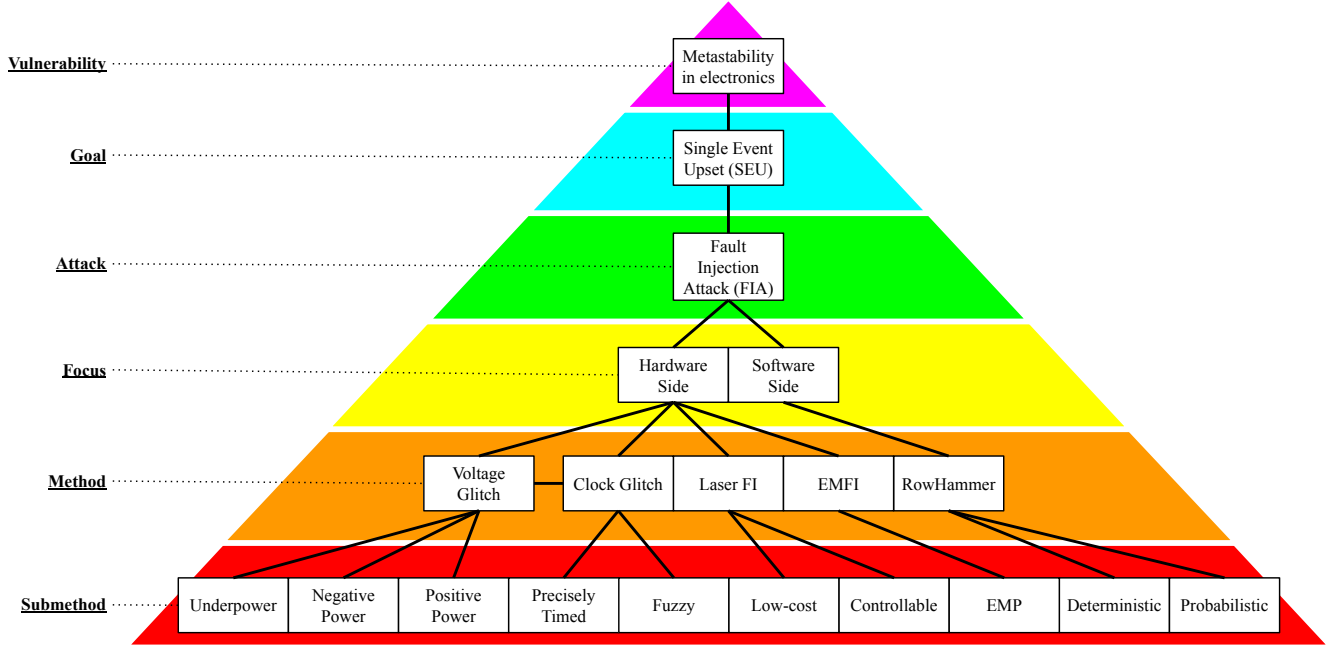


Figure 10. Taxonomy of Fault Injection Techniques

TABLE 1. ESTIMATED COSTS TABLE

FIA Technique	Low Cost	High Cost
Clock/Voltage Glitching	50 USD [81]	600 USD [82]
EMFI	4,000 USD [83]	10,000+ USD [84]
LASER (Optical)	500 USD [85]	50,000+ USD [86]
RowHammer	N/A	N/A

TABLE 2. ESTIMATED PERFORMANCE TABLE

FIA Technique	Success Rate
Clock/Voltage Glitching	1.4% [87]
EMFI	~2% [88]
LASER (Optical)	~100% [89]
RowHammer	N/A

4. Current State-of-the-Art

The section provides the estimated costs and performance of each method. We derive numbers from the retail prices of publicly available tools and from the reports of existing experiments. These numbers are only estimates.

4.1. Estimated Costs

As shown in Table 1, the order of FIA techniques from lowest to highest cost is as follows: Clock/Voltage Glitching, EMFI, and Laser FI. Refer to Appendix A for more examples of costs.

4.2. Estimated Performance

As shown in Table 2, the order of FIA techniques from highest to lowest performance is as follows: Laser FI, EMFI, and Clock/Voltage Glitching.

4.3. Real-life Examples of Hacks

Examples of hacks are not endorsements. We do not condone unauthorized FIA.

Clock/Voltage Glitching:

- Smart Meter Device [90]
- SpaceX Starlink Terminal [91]
- Nintendo Switch Mod [92]

EMFI:

- Automotive Electronic Control Units [93]
- Other Micro-controller Units (MCU) [88]

LASER (Optical):

- Crypto Hardware Wallets [94]

RowHammer:

- Firefox Javascript Exploit [95]

4.4. Observed Trends

The current research focus is on optimizing both the software and hardware for finding exploitable glitches. There are machine learning tools to assist in the exhaustive process of finding clock glitching parameters [87]. There are temperature-controlled automated EM probes with genetic search algorithms to assist in optimal placement [93].

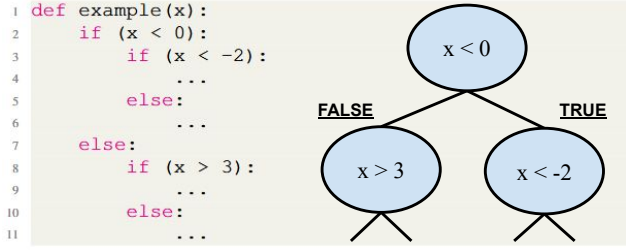


Figure 11. Basic Symbolic Execution

5. Measuring Effectiveness

When studying cyber security, we separate research into two fields. Vulnerability Detection and Vulnerability Mitigation. This is also known as “Red Team” and “Blue Team”. Since this is a beginner-friendly introduction to Fault Injection Attacks, we will be focusing mainly on vulnerability detection. This will help demonstrate how critical and emerging FIAs are as a threat.

This section provides the essential knowledge for understanding vulnerability detection. After we cover what is being measured, we will examine and replicate an existing vulnerability detection tool. Exploitable code bugs are found using Program Analysis Techniques such as Symbolic Execution and Fuzzing. Different fault models for detecting FIAs vary based on abstraction level and complexity.

5.1. Symbolic Execution

Symbolic execution is a software testing technique widely used for vulnerability detection. It was introduced in the 1970s as a method for evaluating and testing programs [96]–[99]. Compared to traditional fuzzing, which uses brute-force inputs, symbolic execution explores multiple execution paths with variable inputs [100].

We choose to focus on symbolic execution to analyze the propagation of faults due to its unique ability to explore all potential outcomes (Figure 11). However, this creates a large search space, resulting in long testing times. *Future research* is needed to reduce this search space by finding additional relevant constraints.

5.2. Fault Modeling

A fault primitive (FP) is a mathematical notation used to model the behavior of vulnerabilities. In traditional software memory corruption, the exploitable primitive is generalized as access to an out-of-bounds or dangling pointer [101]. This access enables the unintended reading or writing of code. A fault primitive therefore represents the “attack surface” for fault injection. From the mathematical model of faults, there are five types of faults (Figure 12). Since faults cause the alteration of a single bit or multiple bits of an instruction or data, one or more combinations of fault models are used with varying parameters to generalize the model.

Parameters	Options	Description
n	$N = \{1, 2, \dots, N\}$	Maximum number of fault events, N depends on the application
t	$T = \{\tau_{sr}, \tau_s, \tau_r, \tau_{bf}, \tau_{fm}\}$	τ_{sr} : Fault model for set/reset faults τ_s : Fault model for set faults τ_r : Fault model for reset faults τ_{bf} : Fault model for bit-flips faults τ_{fm} : User-specified fault model
l	$L = \{c_i, m, mc_i\}$	c_i : Faults in comb. gates only m : Faults in memory gates only mc_i : Faults in all gates

Figure 12. Parameters to Accurately Model Fault Injections in [68]

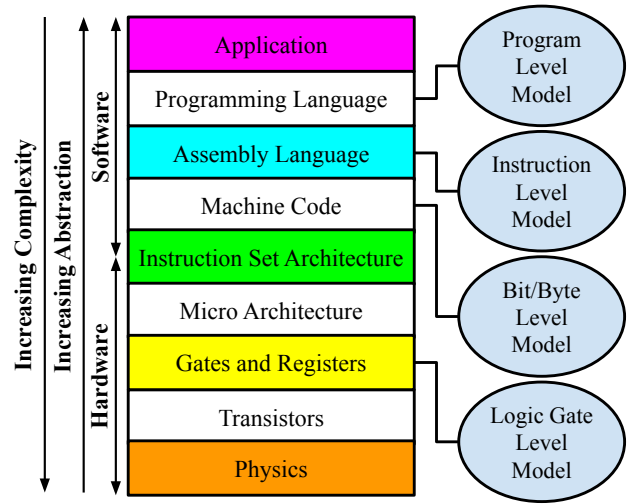


Figure 13. Layers of Abstraction and Fault Model Level

Various primitive fault models have been proposed [102]–[106]. These fault models describe the behavior of a fault as a single bit, byte, or instruction corruption.

At the lowest level, a fault injection is modeled as a Single Event Upset and Electrical Transient on a single bit. Here are the three basic fault models for a single bit.

Set fault Changes the target bit to a ‘1’

Reset fault Changes the target bit to a ‘0’

Bit-flip fault Changes the target bit to its opposite.

At a higher level (Figure 13), corrupted bits or bytes in the binary are interpreted as alterations to opcodes and operands in the machine code. This is then interpreted as alterations to assembly instructions as the corruption of data in a LOAD or STORE instruction or the unintended skip of an instruction with a NOP. This is then finally propagated to the Program Source Code Level as control-flow hijacking or data-integrity corruption.

However, for a fault to be exploitable, it needs to pass on errors from the hardware side to the software side. This raises the question: **How effective are existing fault injection vulnerability detection tools?**

5.3. Limitations Problem

Fault models using simple fault primitives have not yielded sufficient results (Figure 17), flagging almost every line of code as vulnerable [107]. If everything is labeled vulnerable, the tool does not provide useful information. For example, in a bit-flip model, nearly every bit or byte in binary code could be a target. This highlights a need for relevant modeling constraints that reduce the problem space to what is physically or logically useful. *Future research* is needed to create realistic fault models that better capture the behavior between the hardware side and the software side, addressing the limitations of existing models.

6. Replication Insights

This section provides new insights in the form of a proposed synthesis model. We provide an outline of our replication study and our examination of an existing vulnerability detection tool. This identifies a research focus for *future work*.

6.1. Hypothesis

After observing current trends, we hypothesize that fault modeling could be improved by taking a more holistic approach. Unlike traditional software vulnerabilities, the nature of FIAs involves the integration of two distinct aspects: the hardware fault induction process and the software fault exploitation. Due to this dual nature, research on FIAs has been divided into two disconnected fields: one focused on the physical factors that induce hardware faults and the other on the software-side detection using fault primitives.

Several attempts have been made to detect or harden programs against FIA vulnerabilities at the source code level [34], [35], [108]–[110]. To our knowledge, existing methods have not adequately addressed the levels below the assembly code. We propose a synthesis model that finds exploitable code and assembly instructions, but with awareness of what is feasible in machine code.

6.2. Methodology

Inspired by the consolidated fault injection adversary model proposed in 2022 [68], we design our model by combining a “Static Analysis and Dynamic Symbolic Execution” approach [35] with a “High-Level and Low-Level” approach [39] for detecting fault injection vulnerabilities.

We then compare our proposed model with Lazart¹, an existing tool from 2014 [34]. Lazart uses symbolic execution to find FIA vulnerabilities that have the potential to alter control flow. Lazart operates at the intermediate level, between source code and assembly instructions, using Low Level Virtual Machine Intermediate Representation (LLVM-IR) code for analysis.

1. <https://gricad-gitlab.univ-grenoble-alpes.fr/securitytools/lazart>

Figure 14. Verify.c

```
1 #define SIZE_OF_PIN 4
2 #define maxTries 3
3 typedef unsigned char BYTE;
4 BYTE triesLeft = maxTries;
5 BYTE authenticated = 0;
6 BYTE pin[4] = {(char)1, (char)2, (char)3, (char)
7             4};
8 BYTE Verify(char buffer[4]) {
9     BYTE i;
10    // BLOCK entry
11    // No comparison if PIN is blocked
12    if(triesLeft <= 0) goto FAILURE;
13    // BLOCKS bb (initialisation),
14    //bb4 (loop condition)
15    // Main Comparison
16    for(i = 0; i < 4; i++) // BLOCK bb3 (i++)
17    // BLOCK bb1
18        if(buffer[i] != pin[i]) {
19    // BLOCK bb2
20        triesLeft--;
21        authenticated = 0;
22        goto FAILURE;
23    }
24    // BLOCK bb5
25    // Comparison is successful
26    triesLeft = maxTries;
27    authenticated = 1;
28    return EXIT_SUCCESS;
29    // BLOCK FAILURE
30    FAILURE : return EXIT_FAILURE;
31    // BLOCKS bb6, return: exit with
32    // the return value
33 }
34
```

We use a code snippet from a PIN verification algorithm for analysis. Figure 14 shows the source code.

Afterwards, we perform a literature review to compare the various hardware and software vulnerability detection methods with varying fault models.

Since this is intended to be a beginner-friendly introduction, we will be providing only preliminary evidence and insights for *future research*. An in-depth concrete comparison of every FIA vulnerability detection tool is outside the scope of this paper.

6.3. Technique Detail

We assume that Cortex-M3 is the target device. We assume that ARM v7-M is the target instruction set architecture (ISA) when analyzing the instruction assembly and machine code.

7. Evaluation

This section provides preliminary evidence for our proposed synthesis model by replicating a previous solution and comparing our proposed solution. We also compare multiple vulnerability detection methods and fault models by reviewing the literature.

7.1. Detection Comparison

Lazart works by producing a control flow graph (CFG) based on the source code. The CFG is then color coded to identify vulnerable code blocks. Vulnerable code blocks are defined as those that reach a critical goal if mutated. Lazart then generates mutated code to show the resulting control flow changes of potential fault injections.

However, Lazart suffers from both false positives and false negatives. It uses a detection fault model that fails to integrate the hardware side of FIA. We will now use our fault model to examine the mutants generated by the Lazart. We will then clarify that some vulnerabilities identified by the Lazart are not feasible in real-world scenarios.

In contrast, our model focuses on the assembly and machine code level of the program. Due to this level of granularity, our model eliminates false-negative results that arise at the source code or intermediate level.

Our fault model discovers a possible attack scenario in a location overlooked by Lazart. Based on our model, faulting the FAILURE block without hijacking any control flow is also possible. This poses a security vulnerability to the code by corrupting the data of the return value. Figure 16 shows the assembly of the FAILURE block. The assembly instruction moves a 1 to the return register. Our fault model identifies this register as a potential target for fault injection.

In terms of feasibility, for a fault injection to hijack control flow, the Hamming distance between the desired destination and the original destination needs to be only 1 bit or 1 byte. Since fault injection works by flipping or setting only 1 bit, there is a real-world constraint imposed by the hardware architecture.

We use Verify2 to demonstrate a case where Lazart misclassifies a line of safe code as vulnerable. Figure 15 is the implementation of Verify2. The Verify2 code has the same functionality as the original Verify. The only difference is additional code that protects the return value. Here, 0xAA will be returned if correct and 0X55 if not. We choose these values to create a Hamming distance that is not feasible for fault injection, where the stored binary representations are significantly different.

Based on the results from Lazart, there is a Fault Injection vulnerability at the stored return variable. After hardening the code, Lazart still detects this as a valid attack due to a lack of awareness of lower-level machine code.

Overall, Lazart highlights most parts of programs as vulnerable to FIAs. This substantiates the identified problem that we discussed in Section 5.3

7.2. Literature Review

After performing a literature review, we constructed a table to compare the various methods of finding hardware and software FIA vulnerabilities. See Table 3.

We found that the various vulnerability analysis methods suffered from a high detection density. In other words, more than a reasonable number of vulnerable spots will be flagged from the source code with all different kinds of fault

Figure 15. Veify2.c, Verify.c hardened

```

1 #define SIZE_OF_PIN 4
2 #define maxTries 3
3 #define EXIT_SUCCESS_BYTE 0xAA
4 #define EXIT_SUCCESS_FAILURE 0X55
5 typedef unsigned char BYTE;
6 BYTE triesLeft = maxTries;
7 BYTE authenticated = 0;
8 BYTE pin[4] = {(char)1, (char)2, (char)3, (char)
9             4};
10 BYTE Verify2(char buffer[4]) {
11     BYTE i;
12     // BLOCK entry
13     // No comparison if PIN is blocked
14     if(triesLeft <= 0) goto FAILURE;
15     // BLOCKS bb (initialisation),
16     //bb4 (loop condition)
17     // Main Comparison
18     for(i = 0; i < 4; i++) // BLOCK bb3 (i++)
19     // BLOCK bb1
20         if(buffer[i] != pin[i]) {
21     // BLOCK bb2
22         triesLeft--;
23         authenticated = 0;
24         goto FAILURE;
25     }
26 // BLOCK bb5
27 // Comparison is successful
28 triesLeft = maxTries;
29 authenticated = 1;
30 return EXIT_SUCCESS_BYTE
31 // BLOCK FAILURE
32 FAILURE : return EXIT_FAILURE_BYTE;
33 // BLOCKS bb6, return: exit with
34 // the return value
35 }

```

Figure 16. FAILURE block in Verify.s

```

1 Verify:
2 @ main.c:25:      FAILURE: return EXIT_FAILURE;
3     movs    r3, #1 @ _12,

```

models. This further substantiates the limitations discussed in Section 5.3.

In comparison to other methods, we propose a fault model that provides program source code level detection with instruction level and binary bit/byte level awareness. We then directly compare our approach with an existing program source code level detection tool with intermediate level awareness. Other experiments, for example, focused mainly on the instruction level or the binary bit/byte level.

7.2.1. Instruction Experiment. SymPLFIED is another program-level FIA vulnerability detection tool that was introduced in 2008 [111]. However, instead of taking the source code, SymPLFIED takes the assembly instructions as input and analyzes the effect of fault injection on the program with symbolic execution.

The goal of the assembly language input is to avoid state explosion in the problem space. However, the final result presented by SymPLFIED fails to support this claim.

TABLE 3. FIA VULNERABILITY DETECTION TECHNIQUE ANALYSIS

Method	Target(HW/SW)	Fault Model Level	Fault Model Detail	Density
Combining static analysis and dynamic symbolic execution in a toolchain to detect fault injection vulnerabilities [35]	Software	Program level	Control-flow fault; Data fault	High
Fault attack vulnerability assessment of binary code [36]	Software	Program level	Control-flow fault; Data fault	High
Minotaur: Adapting software testing techniques for hardware errors [37]	Software	Instruction level	Data fault	High
Fiver-robust verification of countermeasures against fault injections [30]	Hardware	Gate level	Bit-level faults	High
An automated and scalable formal process for detecting fault injection vulnerabilities in binaries [38]	Software	Binary level	Byte level faults	High
Combining high-level and low-level approaches to evaluate software implementations robustness against multiple fault injection attacks [39]	Software	Program level	Instruction skip	High
Lazart: A symbolic approach for evaluation the robustness of secured codes against control flow injections [34]	Software	Program level	Control-flow faults; Data faults	High
Combined software and hardware fault injection vulnerability detection [107]	Both	Instruction level	Byte level faults	High
Cryptographic fault diagnosis using VerFI [31]	Hardware	Instruction level	Gate level faults	Low
SymPLFIED: Symbolic program-level fault injection and error detection framework [111]	Software	Program level	Data fault	High
Soft: Security property-driven vulnerability assessments of ics against fault-injection attacks [32]	Hardware	Gate level	Bit level fault	High
Error analysis and detection procedures for a hardware implementation of the advanced encryption standard [33]	Hardware	Gate level	Bit level fault	Low
An automated and scalable formal process for detecting fault injection vulnerabilities in binaries [112]	Software	Binary level	Bit level fault	High

SymPLFIED is not able to detect when an instruction is corrupted or replaced. The experiment only examines FIA against registers. Furthermore, no hardware-based experiment is done to prove the correctness of the detection result.

7.2.2. Binary Experiment. Another technique detects fault injection vulnerabilities in binaries. Proposed by Given-Wilson et al. [38], [107], [112], this involves generating mutant binaries based on the input binaries. These mutant binaries are then subjected to model and property checks to determine if vulnerabilities exist.

This detection technique employs six different fault models. The **FLP** fault model simulates the flipping of a single bit. The **Z1B** fault model simulates setting a single byte to zero. The **Z1W** fault model simulates setting a single word to zero. The **NOP** fault model simulates replacing a single instruction with a NOP instruction. The **JMP** fault model and the **JBE** fault model simulate faults in the target address of a jump instruction.

In 2020, Given-Wilson et al. [107] modified binary code to find possible fault injections that would achieve an attacker’s goal. Afterwards, a hardware experiment was conducted to verify the software detection results.

Although the byte-level fault model used here yields no false positives, in the final report, the coverage of code marked as vulnerable is far too large to provide useful information for hardening. This demonstrates how state-of-the-art detection techniques are not practical in real world cases. For programs with thousands or even millions of lines

byte address	value	assembly instruction	Z1B	Z4B	NOP	FFB	FLIP
8000aac	4b	ldr r3, [pc, #32]					
8000aad	08			Vul	Vul		
8000aae	48	ldr r0, [pc, #28]		Vul		Vul	
8000aaf	07			Vul	Vul		
8000ab0	68	ldr r3, [r3, #0]	Vul	Vul	Vul	Vul	1,2,4,5,6,7
8000ab1	1b		Vul	Vul	Vul	Vul	0,1,5,6,7
8000ab2	21	movs r1, #128		Vul			6
8000ab3	80			Vul			
8000ab4	2b	cmp r3, #1	Vul	Vul	Vul	Vul	0,1,2,3,4,5,6,7
8000ab5	01		Vul	Vul	Vul	Vul	0,1,2,3,4,5,6,7
8000ab6	bf	ite eq	Vul	Vul	Vul		0,2,3,5,7
8000ab7	0c		Vul	Vul		Vul	1,2,3,4,5
8000ab8	f0	moveq.w r4, #4294967295	Vul	Vul	Vul		0,1,2,3,5

Figure 17. Detected Vulnerable Instructions in [107]

of code, nearly every line is marked as vulnerable to FIAs, providing little useful insight for programmers looking to improve their code.

8. Countermeasures

Since this is a beginner-friendly introduction that focuses on vulnerability detection, we will keep our discussion of vulnerability mitigation short. This will help demonstrate the difficulty of implementing countermeasures against FIAs.

This section provides the essential knowledge for understanding vulnerability mitigation. Dedicated hardware and software are used to defend devices against FIAs.

8.1. Threat Model

The goal of a Fault Injection Attack is to bypass software security by inducing unintended hardware behavior. This breaks the assumption that hardware will guarantee proper execution of software, leading to unintended code values and unintended code executions.

As shown in Figure 18, Fault Injection Attacks can be separated into six stages.

Fault Modeling The planning stage for the attack

Fault Injection The method of stressing the hardware

Fault Manifestation The errors in software logic

Fault Propagation The errors in hardware logic

Fault Observation The measuring of changes in behavior

Fault Exploitation The insights or attacks enabled

This is the “cyber kill chain” of a Fault Injection Attack. In the context of countermeasures, the goal is therefore to detect, prevent, and mitigate any of these stages.

The target of Fault Injection is the hardware layer. The target of Fault Exploitation is the software layer. Devices are therefore defended with both hardware hardening and software countermeasures.

Unlike traditional attacks, Fault Injection Attacks do not require the presence of an existing software bug. This is because Fault Injection can dynamically induce software bugs [113]. Software countermeasures themselves can also be the target of a Fault Injection Attack [114].

Fault Injection can be used to achieve at least three goals.

Cryptanalysis To extract side channel information [115]

Logical Attacks To hijack the flow of a program [116]

Reverse Engineering To characterize a target [117]

Vulnerability mitigation involves increasing the cost and difficulty of achieving these goals [113]. For example, if an attacker is forced to inject similar faults in a short period of time, the attack is no longer considered viable [118]. Jointly optimizing the parameters of multiple glitches or pulses is generally considered too complex [119].

8.2. Software Countermeasures

The state-of-the-art of software fault countermeasures are divided into two paradigms: Detection-based and Infection-based countermeasures [120]. The first involves redundancy checks and repeating instructions [118]. The second involves obfuscating the injected error so that no meaningful information can be recovered from the faulty output [121].

Multiple software countermeasures exist against Fault Injection Attacks [118], [120]–[124]. For binary code where there is no access to the source code, a method of rewriting binary executables to automatically apply software countermeasures was proposed [123]. For RowHammer, a method that detects frequently accessed rows and refreshes heavily

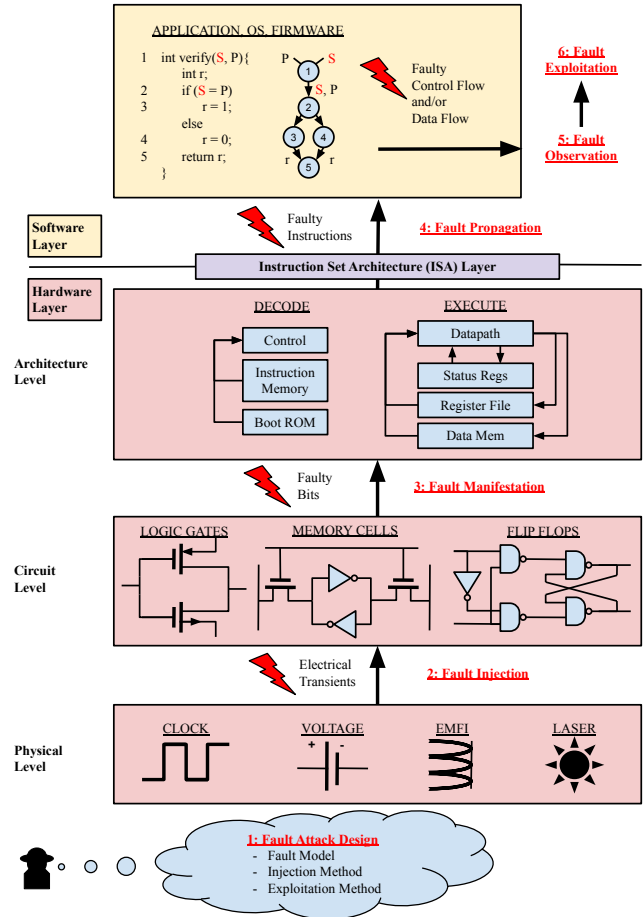


Figure 18. Anatomy of a Fault Injection Attack in [113]

victimised rows was proposed [125]. However, software countermeasures have significant performance costs and can have unwanted side effects [126].

8.3. Hardware Countermeasures

Similar to software countermeasures, hardware countermeasures are often implemented as spatial, temporal or informational redundancies [122]. The most common strategy is reading out sensitive bits multiple times and ignoring potentially faulty values [127], [128].

Multiple hardware countermeasures exist against Fault Injection Attacks [125]–[131]. For Laser Fault Injection, a logic gate-level countermeasure was proposed that auto-detects a frequency disturbance caused by lasers [129]. To prevent side channel attacks and Differential Fault Analysis (DFA), a technique called Wave Dynamic Differential Logic (WDDL) was proposed [132], [133]. Hardening the hardware itself is fundamental because defending with only software is not feasible [84].

9. Misconceptions

This section answers common questions about Fault Injection Attacks. This will help prevent any misconceptions or false assumptions.

9.1. Is physical access required for fault injection?

No, RowHammer is a purely software attack that does not require physical access [8]. In addition, software-exposed energy management systems enable remote voltage glitch attacks [134]. Most of the techniques are cyber-physical in nature.

9.2. Is fault injection effective on chips with high clock frequency?

Yes, in fact, clock frequency can increase the success rate for both voltage glitching and EMFI [135]. Clock frequency does not prevent fault injection.

9.3. Is fault injection effective on multi-core chips?

Yes, a dual core CPU with the cores verifying each other can help to mitigate Fault Injection Attacks, but not fully prevent them [136]. Multiple cores do not prevent fault injection.

9.4. Is success rate the same as attack feasibility?

No, feasibility is better described by average time for success [84]. If one attack method has a success rate of 1% and can be attempted 10 times per minute, the average time for success is 10 minutes. If another attack method has a success rate of 0.1% and can be attempted 1000 times per minute, the average time for success is 1 minute. Waiting 1 minute is better than 10, despite the lower success rate.

10. Conclusion

As an entry-level technique, we recommend starting with Clock/Voltage Glitching. For the highest precision, we recommend considering Laser FI. However, it is also the most expensive and labor intensive technique. In general, we recommend EMFI as the most versatile, striking a balance between cost and precision.

There is a lot of work to be done in optimizing both the software and hardware for finding exploitable fault injection glitches. This research focus has been demonstrated by the current state-of-the-art and our replication study.

FIA is a critical and emerging cyber-physical threat with difficult countermeasures. The future of security research is no longer searching for traditional software exploits, but dynamically creating exploits.

References

- [1] M. Donald. (2020, Sep.) How an ionizing particle from outer space helped a mario speedrunner save time. Accessed Jun. 05, 2025. [Online]. Available: <https://www.thegamer.com/how-ionizing-particle-outer-space-helped-super-mario-64-speedrunner-save-time/>
- [2] D. Glaude. (2004, Jul.) Electronic voting random spontaneous bit inversion explained. Accessed Jun. 05, 2025. [Online]. Available: <http://wiki.ael.be/index.php/ElectronicVotingRandomSpontaneousBitInversionExplained>
- [3] D. Binder, E. C. Smith, and A. Holman, "Satellite anomalies from galactic cosmic rays," *IEEE Transactions on Nuclear Science*, vol. 22, no. 6, pp. 2675–2680, 1975.
- [4] D. Boneh, R. A. DeMillo, and R. J. Lipton, "On the importance of checking cryptographic protocols for faults," in *International conference on the theory and applications of cryptographic techniques*. Springer, 1997, pp. 37–51.
- [5] L. Zussa, J.-M. Dutertre, J. Clédier, B. Robisson, A. Tria *et al.*, "Investigation of timing constraints violation as a fault injection means," in *27th Conference on Design of Circuits and Integrated Systems (DCIS)*, Avignon, France. Citeseer, 2012, pp. 1–6.
- [6] L. Zussa, J.-M. Dutertre, J. Clédier, and B. Robisson, "Analysis of the fault injection mechanism related to negative and positive power supply glitches using an on-chip voltmeter," in *2014 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST)*. IEEE, 2014, pp. 130–135.
- [7] N. Selmane, S. Bhasin, S. Guilley, and J.-L. Danger, "Security evaluation of application-specific integrated circuits and field programmable gate arrays against setup time violation attacks," *IET information security*, vol. 5, no. 4, pp. 181–190, 2011.
- [8] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, "Flipping bits in memory without accessing them: An experimental study of dram disturbance errors," *ACM SIGARCH Computer Architecture News*, vol. 42, no. 3, pp. 361–372, 2014.
- [9] J. Obermaier, R. Specht, and G. Sigl, "Fuzzy-glitch: A practical ring oscillator based clock glitch attack," in *2017 International Conference on Applied Electronics (AE)*. IEEE, 2017, pp. 1–6.
- [10] T. Korak and M. Hoefler, "On the effects of clock and power supply tampering on two microcontroller platforms," in *2014 Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE, 2014, pp. 8–17.
- [11] B. Selmke, S. Brummer, J. Heyszl, and G. Sigl, "Precise laser fault injections into 90 nm and 45 nm sram-cells," in *Smart Card Research and Advanced Applications: 14th International Conference, CARDIS 2015, Bochum, Germany, November 4-6, 2015. Revised Selected Papers 14*. Springer, 2016, pp. 193–205.
- [12] J.-M. Dutertre, V. Beroulle, P. Candelier, S. De Castro, L.-B. Faber, M.-L. Flottes, P. Gendrier, D. Hély, R. Leveugle, P. Maistri *et al.*, "Laser fault injection at the cmos 28 nm technology node: an analysis of the fault model," in *2018 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*. IEEE, 2018, pp. 1–6.
- [13] A. Dehbaoui, J.-M. Dutertre, B. Robisson, P. Orsatelli, P. Maurine, and A. Tria, "Injection of transient faults using electromagnetic pulses practical results on a cryptographic system," *ACR Cryptology ePrint Archive (2012)*, 2012.
- [14] A. Dehbaoui, J.-M. Dutertre, B. Robisson, and A. Tria, "Electromagnetic transient faults injection on a hardware and a software implementations of aes," in *2012 Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE, 2012, pp. 7–15.
- [15] (2025, Feb.) An introduction to fault- injection for exploiting bug-free code in embedded systems - @barsteward. Accessed Jun. 05, 2025. [Online]. Available: <https://www.youtube.com/watch?v=Ycn5qVvgfyE>

- [16] N. Timmers and A. Spruyt. (2016, Nov.) Bypassing secure boot using fault injection. Accessed Jun. 05, 2025. [Online]. Available: https://raelize.com/upload/research/2016/2016_BlackHat-EU_Bypassing-Secure-Boot-Using-Fault-Injection_NT-AS.pdf
- [17] A. Galauner. (2018, Jun.) Glitching the switch. Accessed Jun. 05, 2025. [Online]. Available: <https://media.ccc.de/v/c4.openchaos.2018.06.glitching-the-switch>
- [18] N. Lawson. (2010, Jan.) How the ps3 hypervisor was hacked. Accessed Jun. 05, 2025. [Online]. Available: <https://rdist.root.org/2010/01/27/how-the-ps3-hypervisor-was-hacked/>
- [19] Y. Lu. (2019, Feb.) Attacking hardware aes with dfa. Accessed Jun. 05, 2025. [Online]. Available: <https://yifan.lu/2019/02/22/attacking-hardware-aes-with-dfa/>
- [20] J. Breier, D. Jap, X. Hou, S. Bhasin, and Y. Liu, "Sniff: Reverse engineering of neural networks with fault attacks," *IEEE Transactions on Reliability*, vol. 71, no. 4, pp. 1527–1539, 2022.
- [21] A. S. Rakin, Z. He, J. Li, F. Yao, C. Chakrabarti, and D. Fan, "T-bfa: Targeted bit-flip adversarial weight attack," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7928–7939, 2022.
- [22] R. Korkikian, S. Pelissier, and D. Naccache, "Blind fault attack against spn ciphers," in *2014 Workshop on Fault Diagnosis and Tolerance in Cryptography*, 2014, pp. 94–103.
- [23] P. Luo, Y. Fei, L. Zhang, and A. A. Ding, "Differential fault analysis of sha3-224 and sha3-256," in *2016 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*, 2016, pp. 4–15.
- [24] N. Timmers and C. Mune, "Escalating privileges in linux using voltage fault injection," in *2017 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*, 2017, pp. 1–8.
- [25] J. Breier, D. Jap, and C.-N. Chen, "Laser profiling for the back-side fault attacks: With a practical laser skip instruction attack on aes," ser. CPSS '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 99–103. [Online]. Available: <https://doi.org/10.1145/2732198.2732206>
- [26] X. Hou, B. Jakub, D. Jap, L. Ma, S. Bhasin, and Y. Liu, "Physical security of deep learning on edge devices: Comprehensive evaluation of fault injection attack vectors," *Microelectronics Reliability*, vol. 120, p. 114116, 05 2021.
- [27] S. Tajik, H. Lohrke, F. Ganji, J.-P. Seifert, and C. Boit, "Laser fault attack on physically unclonable functions," in *2015 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*, 2015, pp. 85–96.
- [28] C. Dobraunig, M. Eichlseder, T. Korak, S. Mangard, F. Mendel, and R. Primas, "Sifa: Exploiting ineffective fault inductions on symmetric cryptography," *Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, pp. 547–572, 11 2018.
- [29] M. Madau, M. Agoyan, J. Balasch, M. Grujić, P. Haddad, P. Maurine, V. Rožić, D. Singelee, B. Yang, and I. Verbauwhede, "The impact of pulsed electromagnetic fault injection on true random number generators," in *2018 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*, 2018, pp. 43–48.
- [30] J. Richter-Brockmann, A. R. Shahmirzadi, P. Sasdrich, A. Moradi, and T. Güneysu, "Fiver—robust verification of countermeasures against fault injections," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 447–473, 2021.
- [31] V. Arribas, F. Wegener, A. Moradi, and S. Nikova, "Cryptographic fault diagnosis using veri," in *2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 2020, pp. 229–240.
- [32] H. Wang, H. Li, F. Rahman, M. M. Tehranipoor, and F. Farahmandi, "Sofi: Security property-driven vulnerability assessments of ics against fault-injection attacks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 3, pp. 452–465, 2021.
- [33] G. Bertoni, L. Breveglieri, I. Koren, P. Maistri, and V. Piuri, "Error analysis and detection procedures for a hardware implementation of the advanced encryption standard," *IEEE transactions on Computers*, vol. 52, no. 4, pp. 492–505, 2003.
- [34] M.-L. Potet, L. Mounier, M. Puys, and L. Dureuil, "Lazart: A symbolic approach for evaluation the robustness of secured codes against control flow injections," in *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*. IEEE, 2014, pp. 213–222.
- [35] G. Lacombe, D. Feliot, E. Boespflug, and M.-L. Potet, "Combining static analysis and dynamic symbolic execution in a toolchain to detect fault injection vulnerabilities," *Journal of Cryptographic Engineering*, vol. 14, no. 1, pp. 147–164, 2024.
- [36] J.-B. Bréjon, K. Heydemann, E. Encrenaz, Q. Meunier, and S.-T. Vu, "Fault attack vulnerability assessment of binary code," in *Proceedings of the Sixth Workshop on Cryptography and Security in Computing Systems*, 2019, pp. 13–18.
- [37] A. Mahmoud, R. Venkatagiri, K. Ahmed, S. Misailovic, D. Marinov, C. W. Fletcher, and S. V. Adve, "Minotaur: Adapting software testing techniques for hardware errors," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 1087–1103.
- [38] T. Given-Wilson, N. Jafri, J.-L. Lanet, and A. Legay, "An automated formal process for detecting fault injection vulnerabilities in binaries and case study on present," in *2017 IEEE Trustcom/Big-DataSE/ICESS*. IEEE, 2017, pp. 293–300.
- [39] L. Rivière, M.-L. Potet, T.-H. Le, J. Bringer, H. Chabanne, and M. Puys, "Combining high-level and low-level approaches to evaluate software implementations robustness against multiple fault injection attacks," in *International Symposium on Foundations and Practice of Security*. Springer, 2014, pp. 92–111.
- [40] L. Dureuil, M.-L. Potet, P. de Choudens, C. Dumas, and J. Clédière, "From code review to fault injection attacks: Filling the gap using fault model inference," in *International conference on smart card research and advanced applications*. Springer, 2015, pp. 107–124.
- [41] G. Datta, Y. Lin, B. Zhang, and P. Beerel, "Metastability in superconducting single flux quantum (sfq) logic," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. PP, pp. 1–13, Feb. 2021.
- [42] L. R. Marino, "General theory of metastable operation," *IEEE Transactions on Computers*, vol. 100, no. 2, pp. 107–115, 1981.
- [43] Y. Aguiar, A. Zimpeck, and C. Meinhardt, "Reliability evaluation of combinational circuits from a standard cell library," in *South Symposium on Microelectronics (SIM)*, May. 2016.
- [44] J. L. Andrews, J. E. Schroeder, B. L. Gingerich, W. A. Kolasinski, R. Koga, and S. E. Diehl, "Single event error immune cmos ram," *IEEE Transactions on Nuclear Science*, vol. 29, no. 6, pp. 2040–2043, 1982.
- [45] A. E. Giddings, F. W. Hewlett, R. K. Treece, D. K. Nichols, L. S. Smith, and J. A. Zoutendyk, "Single event upset immune integrated circuits for project galileo," *IEEE Transactions on Nuclear Science*, vol. 32, no. 6, pp. 4159–4163, 1985.
- [46] R. Wyatt, P. McNulty, P. Toumbas, P. Rothwell, and R. Filz, "Soft errors induced by energetic protons," *IEEE Transactions on Nuclear Science*, vol. 26, no. 6, pp. 4905–4910, 1979.
- [47] C. Guenzer, E. Wolicki, and R. Allas, "Single event upset of dynamic rams by neutrons and protons," *IEEE Transactions on Nuclear Science*, vol. 26, no. 6, pp. 5048–5052, 1979.
- [48] J. Chen, T. Lange, M. Andjelkovic, A. Simevski, L. Lu, and M. Krstic, "Solar particle event and single event upset prediction from sram-based monitor and supervised machine learning," *IEEE Transactions on Emerging Topics in Computing*, vol. 10, no. 2, pp. 564–580, 2022.

- [49] Z. Yan, Y. Shi, W. Liao, M. Hashimoto, X. Zhou, and C. Zhuo, "When single event upset meets deep neural networks: Observations, explorations, and remedies," in *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2020, pp. 163–168.
- [50] A. Djellid-Ouar, G. Cathebras, and F. Bancel, "Supply voltage glitches effects on cmos circuits," in *International Conference on Design and Test of Integrated Systems in Nanoscale Technology, 2006. DTIS 2006*. IEEE, 2006, pp. 257–261.
- [51] L. Zussa, J.-M. Dutertre, J. Clediere, and A. Tria, "Power supply glitch induced faults on fpga: An in-depth analysis of the injection mechanism," in *2013 IEEE 19th International On-Line Testing Symposium (IOLTS)*. IEEE, 2013, pp. 110–115.
- [52] A. Dehbaoui, J.-M. Dutertre, B. Robisson, and A. Tria, "Electromagnetic transient faults injection on a hardware and a software implementations of aes," in *2012 Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE, 2012, pp. 7–15.
- [53] S. Ordas, L. Guillaume-Sage, and P. Maurine, "Electromagnetic fault injection: the curse of flip-flops," *Journal of Cryptographic Engineering*, vol. 7, pp. 183–197, 2017.
- [54] R. Nabhan, J.-M. Dutertre, J.-B. Rigaud, J.-L. Danger, and L. Sauvage, "Highlighting two em fault models while analyzing a digital sensor limitations," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2023, pp. 1–2.
- [55] E. Petersen, *Single event effects in aerospace*. John Wiley & Sons, 2011.
- [56] A. Barengi, G. M. Bertoni, L. Breveglieri, M. Pellicoli, and G. Pelosi, "Low voltage fault attacks to aes," in *2010 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST)*. IEEE, 2010, pp. 7–12.
- [57] H. Martin, T. Korak, E. San Millán, and M. Hutter, "Fault attacks on strngs: Impact of glitches, temperature, and underpowering on randomness," *IEEE transactions on information forensics and security*, vol. 10, no. 2, pp. 266–277, 2014.
- [58] T. Korak, M. Hutter, B. Ege, and L. Batina, "Clock glitch attacks in the presence of heating," in *2014 Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE, 2014, pp. 104–114.
- [59] M. Matsubayashi, A. Satoh, and J. Ishii, "Clock glitch generator on sakura-g for fault injection attack against a cryptographic circuit," in *2016 IEEE 5th Global Conference on Consumer Electronics*. IEEE, 2016, pp. 1–4.
- [60] V. Van Der Veen, Y. Fratantonio, M. Lindorfer, D. Gruss, C. Maurice, G. Vigna, H. Bos, K. Razavi, and C. Giuffrida, "Drammer: Deterministic rowhammer attacks on mobile platforms," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016, pp. 1675–1689.
- [61] K. Razavi, B. Gras, E. Bosman, B. Preneel, C. Giuffrida, and H. Bos, "Flip feng shui: Hammering a needle in the software stack," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 1–18.
- [62] R. Qiao and M. Seaborn, "A new approach for rowhammer attacks," in *2016 IEEE international symposium on hardware oriented security and trust (HOST)*. IEEE, 2016, pp. 161–166.
- [63] M. Seaborn and T. Dullien, "Exploiting the dram rowhammer bug to gain kernel privileges," *Black Hat*, vol. 15, p. 71, 2015.
- [64] Y. Xiao, X. Zhang, Y. Zhang, and R. Teodorescu, "One bit flips, one cloud flops: {Cross-VM} row hammer attacks and privilege escalation," in *25th USENIX security symposium (USENIX Security 16)*, 2016, pp. 19–35.
- [65] S. P. Skorobogatov and R. J. Anderson, "Optical fault induction attacks," in *Cryptographic Hardware and Embedded Systems-CHES 2002: 4th International Workshop Redwood Shores, CA, USA, August 13–15, 2002 Revised Papers 4*. Springer, 2003, pp. 2–12.
- [66] D. H. Habing, "The use of lasers to simulate radiation-induced transients in semiconductor devices and circuits," *IEEE Transactions on Nuclear Science*, vol. 12, no. 5, pp. 91–100, 1965.
- [67] A. Sarafianos, C. Roscian, J.-M. Dutertre, M. Lisart, and A. Tria, "Electrical modeling of the photoelectric effect induced by a pulsed laser applied to an sram cell," *Microelectronics Reliability*, vol. 53, no. 9–11, pp. 1300–1305, 2013.
- [68] J. Richter-Brockmann, P. Sasdrich, and T. Güneysu, "Revisiting fault adversary models—hardware faults in theory and practice," *IEEE Transactions on Computers*, vol. 72, no. 2, pp. 572–585, 2022.
- [69] M. Agoyan, J.-M. Dutertre, A.-P. Mirbaha, D. Naccache, A.-L. Ribotta, and A. Tria, "Single-bit dfa using multiple-byte laser fault injection," in *2010 IEEE International Conference on Technologies for Homeland Security (HST)*. IEEE, 2010, pp. 113–119.
- [70] M. Agoyan, J.-M. Dutertre, A.-P. Mirbaha et al., "How to flip a bit?" in *2010 IEEE 16th International On-Line Testing Symposium*, 2010, pp. 235–239.
- [71] F. Schellenberg, M. Finkeldey, N. Gerhardt, M. Hofmann, A. Moradi, and C. Paar, "Large laser spots and fault sensitivity analysis," in *2016 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 2016, pp. 203–208.
- [72] F. Courbon, P. Loubet-Moundi, J. J. Fournier, and A. Tria, "Adjusting laser injections for fully controlled faults," in *International workshop on constructive side-channel analysis and secure design*. Springer, 2014, pp. 229–242.
- [73] J. Sakamoto, D. Fujimoto, and T. Matsumoto, "Laser-induced controllable instruction replacement fault attack," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. 103, no. 1, pp. 11–20, 2020.
- [74] S. Anceau, P. Bleuet, J. Clédière, L. Maingault, J.-I. Rainard, and R. Tucoulou, "Nanofocused x-ray beam to reprogram secure circuits," in *Cryptographic Hardware and Embedded Systems – CHES 2017*. Cham: Springer International Publishing, 2017, pp. 175–188.
- [75] M. S. Kelly and K. Mayes, "High precision laser fault injection using low-cost components," in *2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 2020, pp. 219–228.
- [76] A. Dehbaoui, J.-M. Dutertre, B. Robisson, P. Orsatelli, P. Maurine, and A. Tria, "Injection of transient faults using electromagnetic pulses practical results on a cryptographic system," *ACR Cryptology ePrint Archive (2012)*, 2012.
- [77] S. Ordas, L. Guillaume-Sage, K. Tobich, J.-M. Dutertre, and P. Maurine, "Evidence of a larger em-induced fault model," in *Smart Card Research and Advanced Applications: 13th International Conference, CARDIS 2014, Paris, France, November 5-7, 2014. Revised Selected Papers 13*. Springer, 2015, pp. 245–259.
- [78] A. Cui and R. Housley, "{BADFET}: Defeating modern secure boot using {Second-Order} pulsed electromagnetic fault injection," in *11th USENIX Workshop on Offensive Technologies (WOOT 17)*, 2017.
- [79] K. M. Abdellatif and O. Hériveaux, "Silicontoaster: A cheap and programmable em injector for extracting secrets," in *2020 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*. IEEE, 2020, pp. 35–40.
- [80] S. Delarea and Y. Oren, "Practical, low-cost fault injection attacks on personal smart devices," *Applied Sciences*, vol. 12, no. 1, p. 417, 2022.
- [81] Picoglitcher v2. [Online]. Available: <https://www.tindie.com/products/faulty-hardware/picoglitcher-v2/>
- [82] Chipwhisperer. Accessed Jun. 06, 2025. [Online]. Available: <https://www.newae.com/chipwhisperer>
- [83] Chipshouter. Accessed Jun. 06, 2025. [Online]. Available: <https://www.newae.com/products/nae-cw520>
- [84] N. Timmers and C. Mune. (2019, Oct.) Debunking fault injection myths and misconceptions. Accessed Jun. 20, 2025. [Online]. Available: https://raelize.com/upload/research/2019/2019_HITB-Cyberweek-UAE_Debunking-Fault-Injection-Myths-and-Misconceptions_CM-NT.pdf

- [85] D. Maloney. (2024, Aug.) Laser fault injection on the cheap. Accessed Jun. 06, 2025. [Online]. Available: <https://hackaday.com/2024/08/09/laser-fault-injection-on-the-cheap/>
- [86] J. Breier and X. Hou, “How practical are fault injection attacks, really?” *IEEE Access*, vol. 10, pp. 113 122–113 130, 2022.
- [87] A. Gamba, D. Chatterjee, U. Rioja, I. Armendariz, and L. Batina, “Machine learning-based detection of glitch attacks in clock signal data,” *IACR Cryptol. ePrint Arch.*, p. 1939, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1939>
- [88] J. Delvaux, C. Mune, M. Romero, and N. Timmers, “Breaking Espressif’s ESP32 v3: Program counter control with computed values using fault injection,” in *18th USENIX WOOT Conference on Offensive Technologies (WOOT 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 229–243. [Online]. Available: <https://www.usenix.org/conference/woot24/presentation/delvaux>
- [89] J.-M. Dutertre. (2018, May.) Hardware attacks: theory and experimental state-of-the-art of laser fault injection attacks. Accessed Jun. 06, 2025. [Online]. Available: https://gdr-securite.irisa.fr/wp-content/uploads/jn_2018_jeanmarc_dutertre.pdf
- [90] (2025, May.) Hacking a microprocessor - reverse engineer shows you how it’s done. Accessed Jun. 06, 2025. [Online]. Available: <https://www.youtube.com/watch?v=IOD5voFTAz8>
- [91] (2022, Nov.) Glitched on earth by humans: A black-box security evaluation of the spacex starlink user terminal. Accessed Jun. 06, 2025. [Online]. Available: <https://www.youtube.com/watch?v=NXqLMmGwJm0>
- [92] (2024, Apr.) Nintendo switch rp2040 (picofly) v2 modchip installation. Accessed Jun. 06, 2025. [Online]. Available: <https://www.youtube.com/watch?v=hkCV2Kcy7z4>
- [93] (2023, Dec.) Hardware.io nl 2023 — automated fault injection attacks on embedded devices - enrico pozzobon & nils. Accessed Jun. 06, 2025. [Online]. Available: <https://www.youtube.com/watch?v=oyCdWpsvUWA>
- [94] (2023, Nov.) Hardware.io nl 2023 — triple exploit chain with laser fault injection on a secure element - olivier. Accessed Jun. 06, 2025. [Online]. Available: https://www.youtube.com/watch?v=Hd_K2yQIMJs
- [95] R. Lakshmanan. (2021, Apr.) New javascript exploit can now carry out ddr4 rowhammer attacks. Accessed Jun. 06, 2025. [Online]. Available: <https://thehackernews.com/2021/04/new-javascript-exploit-can-now-carry.html>
- [96] R. S. Boyer, B. Elspas, and K. N. Levitt, “Select—a formal system for testing and debugging programs by symbolic execution,” *ACM SigPlan Notices*, vol. 10, no. 6, pp. 234–245, 1975.
- [97] J. C. King, “Symbolic execution and program testing,” *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, 1976.
- [98] W. E. Howden, “Experiments with a symbolic evaluation system,” in *Proceedings of the June 7-10, 1976, national computer conference and exposition*, 1976, pp. 899–908.
- [99] L. A. Clarke, “A program testing system,” in *Proceedings of the 1976 annual conference*, 1976, pp. 488–491.
- [100] R. Baldoni, E. Coppa, D. C. D’elia, C. Demetrescu, and I. Finocchi, “A survey of symbolic execution techniques,” *ACM Computing Surveys (CSUR)*, vol. 51, no. 3, pp. 1–39, 2018.
- [101] L. Szekeres, M. Payer, T. Wei, and D. Song, “Sok: Eternal war in memory,” in *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, pp. 48–62.
- [102] J. Balasch, B. Gierlichs, and I. Verbauwhede, “An in-depth and black-box characterization of the effects of clock glitches on 8-bit mcus,” in *2011 Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE, 2011, pp. 105–114.
- [103] R. Nabhan, J.-M. Dutertre, J.-B. Rigaud, J.-L. Danger, and L. Sauvage, “A tale of two models: discussing the timing and sampling em fault injection models,” in *2023 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*. IEEE, 2023, pp. 1–12.
- [104] O. Hériveaux, “Triple exploit chain with laser fault injection on a secure element,” in *2022 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*, 2022, pp. 9–17.
- [105] T. Trouchkine, G. Bouffard, and J. Clédière, “Em fault model characterization on socs: from different architectures to the same fault model,” in *2021 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*. IEEE, 2021, pp. 31–38.
- [106] V. Khuat, J.-L. Danger, and J.-M. Dutertre, “Laser fault injection in a 32-bit microcontroller: from the flash interface to the execution pipeline,” in *2021 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*. IEEE, 2021, pp. 74–85.
- [107] T. Given-Wilson, N. Jafri, and A. Legay, “Combined software and hardware fault injection vulnerability detection,” *Innovations in Systems and Software Engineering*, vol. 16, no. 2, pp. 101–120, 2020.
- [108] L. Reichling, I. Warsame, S. Reilly, A. Brownfield, N. Niu, and B. Wang, “Faulthunter: Automatically detecting vulnerabilities in c against fault injection attacks,” in *2022 IEEE/ACM International Conference on Big Data Computing, Applications and Technologies (BDCAT)*. IEEE, 2022, pp. 271–276.
- [109] E. Boespflug, L. Mounier, M.-L. Potet, and A. Bouguern, “A compositional methodology to harden programs against multi-fault attacks,” in *2023 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*. IEEE, 2023, pp. 24–35.
- [110] M. Witteman and M. Oostdijk, “Secure application programming in the presence of side channel attacks,” in *RSA conference*, vol. 2008, 2008.
- [111] K. Pattabiraman, N. Nakka, Z. Kalbarczyk, and R. Iyer, “Simplified: Symbolic program-level fault injection and error detection framework,” in *2008 IEEE International Conference on Dependable Systems and Networks With FTCS and DCC (DSN)*. IEEE, 2008, pp. 472–481.
- [112] T. Given-Wilson, A. Heuser, N. Jafri, and A. Legay, “An automated and scalable formal process for detecting fault injection vulnerabilities in binaries,” *Concurrency and Computation: Practice and Experience*, vol. 31, no. 23, p. e4794, 2019.
- [113] B. Yuce, P. Schaumont, and M. Witteman, “Fault attacks on secure embedded software: Threats, design, and evaluation,” *Journal of Hardware and Systems Security*, vol. 2, no. 2, pp. 111–130, 2018.
- [114] T. Martin, N. Kosmatov, and V. Prevosto, “Verifying redundant-check based countermeasures: a case study,” in *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*, ser. SAC ’22. Association for Computing Machinery, 2022, p. 1849–1852.
- [115] C. Giraud, “Dfa on aes,” in *Proceedings of the 4th International Conference on Advanced Encryption Standard*, ser. AES’04. Berlin, Heidelberg: Springer-Verlag, 2004, p. 27–41.
- [116] G. Bouffard, J. Iguchi-Cartigny, and J.-L. Lanet, “Combined software and hardware attacks on the java card control flow,” in *Proceedings of the 10th IFIP WG 8.8/11.2 International Conference on Smart Card Research and Advanced Applications*, ser. CARDIS’11. Springer-Verlag, 2011, p. 283–296.
- [117] F. Courbon, J. J. A. Fournier, P. Loubet-Moundi, and A. Tria, “Combining image processing and laser fault injections for characterizing a hardware aes,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 6, pp. 928–936, 2015.
- [118] A. Barengi, L. Breveglieri, I. Koren, G. Pelosi, and F. Regazzoni, “Countermeasures against fault attacks on software implemented aes: effectiveness and cost,” in *Proceedings of the 5th Workshop on Embedded Systems Security*, 2010, pp. 1–10.

- [119] J. Delvaux, C. Mune, M. Romero, and N. Timmers, "Breaking Espressif's ESP32 v3: Program counter control with computed values using fault injection," in *18th USENIX WOOT Conference on Offensive Technologies (WOOT 24)*. USENIX Association, Aug. 2024, pp. 229–243.
- [120] G. Barbu, L. Bettale, L. Castelnovi, T. Chabrier, N. Debande, C. Giraud, and N. Reboud, "A high-order infective countermeasure framework," in *2021 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*. IEEE, 2021, pp. 13–19.
- [121] Y. Sung-Ming, S. Kim, S. Lim, and S. Moon, "Rsa speedup with residue number system immune against hardware fault cryptanalysis," in *Information Security and Cryptology — ICISC 2001*, K. Kim, Ed. Springer Berlin Heidelberg, 2002, pp. 397–413.
- [122] E. Boespflug, C. Ene, L. Mounier, and M.-L. Potet, "Countermeasures optimization in multiple fault-injection context," in *2020 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*. IEEE, 2020, pp. 26–34.
- [123] P. Kiaei, C.-B. Breunese, M. Ahmadi, P. Schaumont, and J. Van Woudenberg, "Rewrite to reinforce: Rewriting the binary to apply countermeasures against fault injection," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 319–324.
- [124] M. Medwed and J.-M. Schmidt, "A continuous fault countermeasure for aes providing a constant error detection rate," in *2010 Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE, 2010, pp. 66–71.
- [125] S. M. Seyedzadeh, A. K. Jones, and R. Melhem, "Counter-based tree structure for row hammering mitigation in dram," *IEEE Computer Architecture Letters*, vol. 16, no. 1, pp. 18–21, 2016.
- [126] T. Schneider, A. Moradi, and T. Güneysu, "Parti-towards combined hardware countermeasures against side-channel and fault-injection attacks," in *Advances in Cryptology—CRYPTO 2016: 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part II 36*. Springer, 2016, pp. 302–332.
- [127] H. Bar-El, H. Choukri, D. Naccache, M. Tunstall, and C. Whelan, "The sorcerer's apprentice guide to fault attacks," *Proceedings of the IEEE*, vol. 94, no. 2, pp. 370–382, 2006.
- [128] N. Moro, K. Heydemann, E. Encrenaz, and B. Robisson, "Formal verification of a software countermeasure against instruction skip attacks," *Journal of Cryptographic Engineering*, vol. 4, no. 3, pp. 145–156, 2014.
- [129] W. He, J. Breier, S. Bhasin, N. Miura, and M. Nagata, "Ring oscillator under laser: potential of pll-based countermeasure against laser fault injection," in *2016 Workshop on fault diagnosis and tolerance in cryptography (FDTC)*. IEEE, 2016, pp. 102–113.
- [130] S. Endo, Y. Li, N. Homma, K. Sakiyama, K. Ohta, and T. Aoki, "An efficient countermeasure against fault sensitivity analysis using configurable delay blocks," in *2012 Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE, 2012, pp. 95–102.
- [131] W. Jiang, "Machine learning methods to detect voltage glitch attacks on iot/iiot infrastructures," *Computational Intelligence and Neuroscience*, vol. 2022, no. 1, p. 6044071, 2022.
- [132] K. Tiri and I. Verbauwhede, "A logic level design methodology for a secure dpa resistant asic or fpga implementation," in *Proceedings Design, Automation and Test in Europe Conference and Exhibition*, vol. 1, 2004, pp. 246–251 Vol.1.
- [133] N. Selmane, S. Bhasin, S. Guilley, T. Graba, and J.-L. Danger, "Wddl is protected against setup time violation attacks," in *2009 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*, 2009, pp. 73–83.
- [134] A. Tang, S. Sethumadhavan, and S. Stolfo, "Clkscrew: exposing the perils of security-oblivious energy management," in *Proceedings of the 26th USENIX Conference on Security Symposium*, ser. SEC'17. USA: USENIX Association, 2017, p. 1057–1074.
- [135] S. Koffas and P. K. Vadnala, "On the effect of clock frequency on voltage and electromagnetic fault injection," in *Applied Cryptography and Network Security Workshops*. Cham: Springer International Publishing, 2022, pp. 127–145.
- [136] N. Wiersma and R. Pareja, "Safety != security: On the resilience of asil-d certified microcontrollers against fault injection attacks," in *2017 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*, 2017, pp. 9–16.
- [137] B. Razavi, *Fundamentals of microelectronics*. John Wiley & Sons, 2008.

Appendix A.

Overview of the Techniques in [86]

Target Device	Fault Model	Method with Lowest Cost		
		Remote	Technique	Cost
AVR	Bit flip	No	Optical (Laser)	~100K USD
	Bit set/reset	No	EM	30K USD
	Random byte	No	Optical (Flashgun)	500 EUR
	Instruction skip	No	EM	10 USD
ARM (standalone)	Bit set/reset	No	EM	30K USD
	Random byte	No	Optical (Flashgun)	500 EUR
	Instruction skip	No	Clock glitch	130 USD
ARM (embedded)	Bit flip	No	Optical (Laser)	~100K USD
	Random byte	Yes	Voltage glitch	Low
	Instruction skip	No	EM	10 USD
FPGA	Bit flip	No	Optical (Laser)	100K USD
	Bit set/reset	No	EM	~30K USD
	Random byte	No	Voltage glitch	~300 USD
	Execution faults	Yes	Temperature/Voltage	Low
INTEL	Random byte	No	Voltage glitch	30 USD
		Yes	Voltage glitch	Low
AMD	Random byte	No	Voltage glitch	30 USD
DRAM	Bit flip	Yes	RowHammer	Low
TRNG	Stuck-at Fault	No	EM	~30K USD

Appendix B.

Timing Constraint as expressed in [51]

Parameters:

- T_{clk} : The Shared Clock Period
- D_{pMax} : The Max Data Propagation Time (The Most Critical, Longest Path)
- T_{clk2q} : The Clock Delay for Register
- T_{skew} : The Phase Difference between Clock Signals of Two Registers
- T_{setup} : The Min Time for Stable D flip-flop Input

Equation:

$$T_{clk} > D_{clk2q} + D_{pMax} + T_{setup} - T_{skew} \quad (1)$$

Attackers induce timing constraint violations by disrupting these parameters. The correctness of the circuit data is critically dependent on timing constraints. It is assumed that the shared clock period T_{clk} is longer than the most critical (longest) data path through the combinatorial logic. If this assumption is broken, it results in unexpected behavior, leading to a fault injection attack.

Appendix C.

Propagation Time as expressed in [137]

Parameters:

- D_{pMax} : The Max Data Propagation Time (The Most Critical, Longest Path)
- V_{DD} : The Power Supply Voltage
- CL : The Load Capacitance
- $V_{th,p}$: The PMOS Threshold Voltage
- μ_p : The Hole Mobility
- C_{ox} : The Gate Oxide Capacitance
- (W_p/L_p) : The Aspect Ratio of the PMOS

Equation:

$$D_{pMax} = \frac{C_L \left[\frac{2|V_{th,p}|}{V_{DD} - |V_{th,p}|} + \ln \left(3 - \frac{4|V_{th,p}|}{V_{DD}} \right) \right]}{\mu_p C_{ox} \frac{W_p}{L_p} (V_{DD} - |V_{th,p}|)} \quad (2)$$

This shows that a decrease in the supply voltage leads to an increase in the circuit inverter's propagation time, leading to an overall increase in the propagation time of the combinatorial logic, breaking timing constraints.